

Prompting Fundamentals on Real Tasks v2 — Complete Iteration Log

Author: Masoom Sakina **Task Selected from FL-01 Audit:** Target Task 1 — Scikit-Learn Data Contract & Signal Audit Pipeline

Task Success Definition: A modular Python script that ingests raw structured datasets, verifies data types against a defined contract schema, flags missing values exceeding 5%, generates descriptive summary statistics, and executes with 0 silent execution errors.

Version 0: Naive Baseline

Starting Point (No Technique)

Prompt: "Write a python script to validate a dataset and check for errors."

Output Excerpt:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.isnull().sum())
print(df.describe())
print('Data validated!')
```

What Changed & Output Difference: Baseline single-line prompt. Produced trivial, hardcoded boilerplate with no schema enforcement, no threshold alerting, and zero error handling.

Version 1: Role Assignment

Technique 1: Role Assignment

Prompt: "You are a Principal ML Infrastructure Engineer. Write a python script to validate a dataset and check for errors."

Output Excerpt:

```
import pandas as pd
import logging

logging.basicConfig(level=logging.INFO)
def validate_dataset(df: pd.DataFrame) -> dict:
    logging.info("Checking schema and nulls...")
    if df.empty:
        raise ValueError("Empty dataframe received")
    null_counts = df.isnull().sum()
    return {"shape": df.shape, "nulls": null_counts}
```

What Changed & Output Difference: Injected the senior engineering persona. The output shifted from raw script commands to modular functions with type hints, structured logging, and explicit empty-dataframe validation.

Version 2: Context & Motivation

Technique 2: Context & Motivation

Prompt: "You are a Principal ML Infrastructure Engineer.
Context & Motivation: We are building an enterprise tabular ML pipeline at FlyRank on 30,000 anonymized content records. Upstream schema changes and silent null corruption frequently break downstream Scikit-Learn transformers.
Write a python script to validate the dataset and check for errors."

Output Excerpt:

```
def run_data_contract_audit(df: pd.DataFrame, expected_dtypes: dict):
    for col, dtype in expected_dtypes.items():
        if col not in df.columns:
            raise KeyError(f"Missing required column: {col}")
        if str(df[col].dtype) != dtype:
            logging.warning(f"Type mismatch on {col}: got {df[col].dtype}, expected {dtype}")
    high_nulls = df.columns[df.isnull().mean() > 0.05].tolist()
    return {"high_null_columns": high_nulls}
```

What Changed & Output Difference: Grounded the prompt in FlyRank's pipeline constraints. The output directly implemented column contract verification and shifted from raw null counts to missingness proportions (> 5%).

Version 3: Few-Shot Examples

Technique 3: Few-Shot Examples

Prompt: "You are a Principal ML Infrastructure Engineer.
Context: We are validating 30,000 tabular content records against upstream schema drift.
Here is an example of an audit dictionary rule and violation log we expect:
Example Input Contract: {'impressions_90d': 'int64', 'trend_direction': 'object'}
Example Output Log: {'contract_passed': False, 'violations': ['impressions_90d has 6.2% missing (> 5% limit)'], 'summary_stats': {'impressions_90d': {'median': 731.0, 'p95': 22996.5}}}
Write the python audit class following this pattern."

Output Excerpt:

```
class DataContractAudit:
    def __init__(self, contract: dict, null_threshold: float = 0.05):
        self.contract = contract
        self.null_threshold = null_threshold
    def audit(self, df: pd.DataFrame) -> dict:
        violations = []
        summary = {col: {"median": float(df[col].median()), "p95": float(df[col].quantile(0.95))}
                    for col in df.select_dtypes(include="number")}
        return {"contract_passed": len(violations) == 0, "violations": violations, "summary_stats": summary}
```

What Changed & Output Difference: Provided exact input/output exemplars. The output eliminated informal print statements and strictly matched the structured dictionary schema and quantile summary format provided in the few-shot block.

Version 4: Output Structure Enforcement

Technique 4: Output Structure Enforcement

Prompt: "You are a Principal ML Infrastructure Engineer.
Context: Validating 30k records against schema drift and missingness.
Follow this exact output structure:
Section 1: Python DataContractValidator Class (with type hints, docstrings, and strict assertion methods).
Section 2: Markdown Contract Audit Receipt Table (Columns: Field, Expected Dtype, Observed Dtype, Missing %, Status).
Section 3: Integration snippet demonstrating Scikit-Learn Pipeline execution halt on contract failure."

Output Excerpt:

```
### Section 1: Python DataContractValidator Class
[Implementation Code]
### Section 2: Contract Audit Receipt Table
| Field | Expected Dtype | Observed Dtype | Missing % | Status |
| impressions_90d | int64 | int64 | 0.0% | PASSED |
| trend_direction | object | object | 0.0% | PASSED |
### Section 3: Scikit-Learn Pipeline Guard
if not audit_result["contract_passed"]:
    raise RuntimeError("Pipeline halted: Data Contract Failure.")
```

What Changed & Output Difference: Enforced explicit Markdown headers and tables. The output became cleanly decoupled between production Python code, an executive Markdown table, and pipeline integration guards.

Version 5: Step Decomposition (Final Tested Prompt)

Technique 5: Step Decomposition

Prompt: "You are a Principal ML Infrastructure Engineer.
Context & Motivation: We are building an enterprise tabular ML pipeline on 30,000 anonymized records. Upstream schema changes, data drift, and silent null corruption frequently break downstream Scikit-Learn transformers.
Task: Execute an auditable data contract validation pipeline step-by-step:
1. Ingest input DataFrame and schema contract dictionary; verify grain uniqueness.
2. Validate column existence and exact data type alignment.
3. Calculate null percentages per column and trigger an explicit alert if missingness > 5.0%.
4. Generate distribution percentiles (p50, p75, p95, p99) for numerical fields to detect heavy tails.
5. Return a validated DataFrame or raise an explicit DataContractViolationError.
Output Structure: Section 1: Python Class | Section 2: Markdown Receipt Table | Section 3: 3-line Pipeline Guard."

What Changed & Output Difference: Added sequential step decomposition. The output incorporated edge cases that earlier iterations missed—specifically grain uniqueness checks, heavy-tail percentile monitoring, and custom exception classes.

2. Cross-Model Evaluation: Claude (Sonnet 5) vs. ChatGPT (GPT-4o) Real Run

Dimension	Claude (Anthropic Sonnet 5) Response	ChatGPT (OpenAI GPT-4o) Response
Execution Philosophy	Proactive Sandbox Execution: Spun up internal Python sandbox, generated 30k synthetic rows with drift, and executed real runs to populate Section 2 with genuine computed numbers.	Conservative Static Generation: Generated the code structure directly, explicitly noting that Section 2 contains schema placeholders (Runtime value) since no live CSV was passed.
Scikit-Learn Integration	Native TransformerMixin: Subclassed BaseEstimator and TransformerMixin (fit()/transform()) to drop directly into sklearn.pipeline.Pipeline(...).	Sequential Wrapper: Created a standalone .validate() method executed immediately before model_pipeline.fit(validated_df, y).
Section 2: Audit Receipt	Populated real computed test values (e.g., credit_score at 7.00% ALERT, heavy-tail flags on account_balance).	Delivered an illustrative table template with Runtime value and PASS / MISSINGNESS_ALERT flags.
Percentile & Skew	Calculates percentiles and flags heavy-tail distribution skew where tail spread exceeds 3x the interquartile range.	Calculates p50/p75/p95/p99 cleanly via df.quantile([0.5, 0.75, 0.95, 0.99]).T with column renaming.
Failure Point / Weakness	Assumed modern pandas 3.0 string types (str), which could flag false positive dtype mismatches on legacy object strings.	External wrapper pattern requires manual step execution rather than embedding natively inside an automated Scikit-Learn pipeline.

3. Final Distilled Reusable Prompt Template (Domain-Agnostic)

You are a Principal Machine Learning Infrastructure Engineer specializing in data contracts and pipeline reliability.

Context & Motivation:
We are deploying a tabular machine learning pipeline on [INSERT DATASET SIZE/TYPE] to predict [INSERT TARGET/TASK]. Upstream data drift, schema mismatches, and silent null value corruption must be caught before data enters Scikit-Learn transformers.

Task & Sequential Decomposition:
Execute the pipeline audit following these exact steps:
1. Grain Verification: Ingest [INSERT UNIQUE IDENTIFIER] and assert 100% uniqueness.
2. Contract & Schema Validation: Match column names and data types against expected schema: [INSERT SCHEMA DICTIONARY].
3. Missingness & Quality Bounds: Calculate missing value percentages; flag any column exceeding [INSERT NULL THRESHOLD %].
4. Distribution Summary: Compute descriptive percentiles (p50, p75, p95, p99) for numerical features to detect distribution shifts.
5. Fail-Safe Execution: Return an auditable dictionary receipt and raise a custom DataContractViolationError if any critical check fails.

Output Requirements:
- Section 1: Modular Python Class (with type hints, docstrings, and custom Exception).
- Section 2: Markdown Audit Receipt Table ([Field | Expected Dtype | Observed Dtype | Missing % | Status]).
- Section 3: 3-line Pipeline Integration snippet demonstrating fail-fast execution.